

Universidad ORT Uruguay
Facultad de Ingeniería

Pronóstico calibrado de entrega agéntica

*Sistema de pronóstico probabilístico y evaluación de calibración para
software desarrollado con agentes de IA*

Entregado como requisito para la obtención del título de
Máster en Inteligencia Artificial

Joaquín Guerra - 307854

Germán Pazos - 242016

Ramiro Sanes - 368397

Tutora: Mikaela Pisani

Abstract

El desarrollo de software ejecutado por agentes de inteligencia artificial produce un registro automático del proceso de entrega: qué tarea se ejecutó, cuántas iteraciones de corrección requirió, cuándo intervino una persona y cuánto demoró cada decisión. Este trabajo utiliza ese registro para abordar una pregunta con valor comercial directo y sin respuesta publicada en la literatura revisada: si es posible pronosticar de forma probabilística y calibrada la entrega de software desarrollado con agentes, y qué aporta cada componente del modelo por encima de alternativas simples.

Se propone un sistema que ingiere la telemetría histórica de entrega de la empresa Rootstrap, produce la distribución de probabilidad de la duración de cada tarea y de la fecha de finalización de cada proyecto, y genera de forma automática un boletín de calibración que cuantifica cuánto puede confiarse en cada pronóstico emitido. La evaluación se realiza mediante *backtesting* con origen móvil y métricas de la literatura de pronóstico probabilístico (cobertura de cuantiles, transformada integral de probabilidad y CRPS), y compara el modelo con *baselines* especificados antes de observar los resultados. [PENDIENTE: completar con los resultados principales una vez ejecutada la evaluación.]

Palabras clave: pronóstico probabilístico, calibración, estimación de software, agentes de inteligencia artificial, simulación de Monte Carlo, clases de referencia, *backtesting*.

Índice

[PENDIENTE: Insertar → Índice → Con números de página]

1 Introducción

1.1 Contexto y motivación

La estimación del esfuerzo y de la duración del desarrollo de software cuenta con una literatura extensa, revisada de forma sistemática al menos desde 2007 [1]. Pese a ese volumen de trabajo, la exactitud de las estimaciones sigue siendo limitada. En un experimento con trece profesionales, los intervalos que estos declaraban con 90 % de confianza contuvieron el valor real solo en el 68 % de los casos, pese a haber recibido antes retroalimentación sobre el esfuerzo real de diez tareas de entrenamiento [2]. En un estudio sobre cinco proyectos ágiles, la magnitud media del error relativo (MMRE) de las estimaciones de esfuerzo realizadas por los propios desarrolladores fue de 134 %, aunque los autores advierten que el esfuerzo real se reconstruyó bajo supuestos que podrían inflar esa cifra [3].

Rootstrap, la empresa que aporta los datos de este trabajo, atribuye esa situación a dos causas: el proceso de entrega variaba en cada equipo y los datos que lo describían eran autorreportados [4]. Según la misma fuente, un sistema de entrega ejecutado por agentes elimina ambas causas. El proceso pasa a ser un programa versionado, que ejecuta el mismo ciclo en cada proyecto, y cada paso deja registros con marca de tiempo: los rebotes en control de calidad, las decisiones solicitadas al cliente y la versión del proceso que produjo cada unidad de trabajo [4].

El trabajo ejecutado por agentes ya tiene escala. El conjunto de datos AIDev reúne 932.791 *pull requests* abiertos por cinco agentes de código en 116.211 repositorios públicos hasta agosto de 2025 [5]. La evidencia de campo muestra, a la vez, que ese trabajo no se completa sin intervención humana. En un estudio con 19 desarrolladores profesionales que trabajaron sobre 33 *issues* reales con un agente, el 55 % de los casos válidos terminó con éxito; la tasa fue de 83 % cuando la tarea se descomponía en pasos incrementales y de 38 % cuando se delegaba de una sola vez [6].

Estas condiciones abren la posibilidad de medir de forma sistemática cuánto demora el trabajo ejecutado por agentes y, con ello, de construir pronósticos que puedan contrastarse con lo que efectivamente ocurrió.

1.2 Planteamiento del problema

Rootstrap propone un método de estimación basado en su telemetría. El método define una unidad de trabajo estable, la *capability*, clasificada antes de su ejecución en diez clases de

referencia; un modelo de la duración con variables medidas en lugar de autorreportadas; y una simulación de Monte Carlo sobre el grafo de dependencias del proyecto, cuyo resultado es una distribución de fechas resumida en los cuantiles P50, P80 y P95 [4]. El documento describe también un protocolo de validación basado en la cobertura de cuantiles, la transformada integral de probabilidad y el CRPS, pero no presenta datos empíricos ni resultados de ese protocolo, y cierra señalando que el boletín de calibración, y no la descripción del método, es la evidencia de que sus promesas se sostienen [4].

La utilidad de un pronóstico probabilístico depende de que las probabilidades que declara se correspondan con frecuencias observadas. Si un sistema asigna una probabilidad de 80 % a terminar antes de cierta fecha, esa fecha debería cumplirse en aproximadamente ocho de cada diez casos comparables. Esa propiedad se denomina calibración [7].

La calibración, sin embargo, no alcanza por sí sola. Un sistema que asigna a toda tarea un intervalo muy amplio puede estar bien calibrado y no aportar información para decidir. Gneiting, Balabdaoui y Raftery formalizaron este problema como el principio de maximizar la nitidez (*sharpness*) de las distribuciones predictivas sujeto a su calibración, y mostraron que pronosticadores de calidad muy distinta pueden presentar coberturas prácticamente idénticas [7]. En consecuencia, la evaluación de un sistema de pronóstico debe medir tanto su calibración como la información que aporta.

El problema que aborda este trabajo es la falta de evidencia empírica, independiente y reproducible sobre la calidad de los pronósticos probabilísticos de entrega de software ejecutado por agentes.

1.3 Pregunta de investigación y objetivos

La pregunta que orienta el trabajo es la siguiente: ¿es posible pronosticar de forma probabilística y calibrada la entrega de software ejecutado por agentes, y qué aporta cada componente del modelo por encima de alternativas simples especificadas de antemano?

El objetivo general es construir y validar empíricamente un sistema de pronóstico probabilístico para proyectos de software desarrollados con agentes de inteligencia artificial, cuyo diferencial sea reportar la calidad de su propia calibración. Los objetivos específicos son los siguientes:

- OE1.** Definir un esquema canónico para la telemetría de entrega agéntica (tareas, duraciones, iteraciones de corrección, escalaciones a personas y versión del proceso) e implementar su ingesta desde al menos una fuente real.
- OE2.** Implementar un motor de pronóstico que, a partir de las distribuciones históricas por categoría de tarea y del grafo de dependencias del proyecto, produzca la

distribución de probabilidad de la fecha de finalización, resumida en los cuantiles P50, P80 y P95. La elección del enfoque de estimación se fundamentará en una revisión de las alternativas aplicables.

- OE3.** Construir una capa de evaluación automática de los pronósticos, basada en *backtesting* con origen móvil y en métricas de pronóstico probabilístico: cobertura de cuantiles, transformada integral de probabilidad y CRPS.
- OE4.** Comparar el desempeño del modelo con al menos dos *baselines* especificados de antemano, el pronóstico por conteo de tareas y las distribuciones por categoría sin aprendizaje, para cuantificar el aporte de cada componente.
- OE5.** Estudiar empíricamente al menos una de dos preguntas que condicionan la validez del pronóstico: (a) la confiabilidad entre evaluadores al asignar categorías a las tareas, y (b) cuánta historia conviene utilizar cuando el proceso de entrega cambia entre versiones.

1.4 Contribución

El aporte principal no es el motor de pronóstico, cuyos componentes (la simulación de Monte Carlo sobre datos históricos, las clases de referencia y la actualización bayesiana) cuentan con antecedentes publicados, sino la capa de evaluación. En la literatura revisada no se encontraron evaluaciones de la calibración de distribuciones predictivas sobre la entrega de software, ni con equipos humanos ni con agentes. Existen evaluaciones de la calibración de probabilidades binarias sobre trabajo agéntico, como la curva de calibración que reportan Dao et al. para el esfuerzo de revisión de *pull requests* [8], y evaluaciones con cobertura, transformada integral de probabilidad y CRPS en otros dominios, como el costo de proyectos públicos de gran escala [9]. El cruce entre ambas líneas no aparece en los trabajos relevados.

En concreto, el trabajo aporta:

- un sistema que produce pronósticos de entrega y su propio boletín de calibración a partir de telemetría real;
- una evaluación independiente, con un diseño especificado antes de observar los resultados, del aporte de cada componente del modelo frente a *baselines* con respaldo publicado;
- evidencia empírica sobre la confiabilidad de la clasificación de tareas o sobre el efecto de la no estacionariedad del proceso, según el estudio que se priorice;
- una caracterización descriptiva de la duración de tareas de software ejecutadas por agentes en proyectos comerciales.

Un resultado desfavorable se considera un resultado válido. Si el modelo presenta una calibración pobre, o si los *baselines* simples igualan al modelo completo, el hallazgo se reporta como tal. Existen antecedentes de resultados negativos publicados en estimación de software [10], y el diseño de la evaluación busca que sus conclusiones sean informativas cualquiera sea su dirección.

1.5 Alcance

La unidad primaria de análisis es la tarea y no el proyecto. El histórico disponible comprende alrededor de diez proyectos, lo que impide validar estadísticamente la calibración a nivel de proyecto; a nivel de tarea se espera contar con entre 100 y 300 observaciones. [PENDIENTE: confirmar el volumen con el export de Rootstrap.]

El alcance se organiza en dos niveles. Los resultados comprometidos son el esquema canónico de telemetría con su proceso de ingesta (R1), el motor de pronóstico en su primera versión (R2), el boletín de calibración sobre el histórico con *backtesting* de origen móvil (R3), la comparación contra al menos dos *baselines* con un diseño fijado antes de ver los resultados (R4) y el informe final con los hallazgos (R5). Los resultados deseables, sujetos a la disponibilidad de datos, son la distribución de las horas de intervención humana como segunda variable pronosticada (R6), el segundo estudio del OE5 (R7) y la proyección continua sobre un proyecto en curso (R8).

Quedan fuera del alcance la modelización de riesgos correlacionados entre tareas, las dependencias ocultas muestreadas y la réplica completa del motor descrito por Rootstrap. En diciembre de 2026 se revisará si el volumen y la calidad de los datos sostienen la comparación a nivel de tarea; de no ser así, el alcance se reducirá y el cambio se declarará en este documento.

1.6 Estructura del documento

El capítulo 2 presenta los conceptos teóricos en los que se apoya el trabajo. El capítulo 3 revisa el estado del arte en las cinco líneas de investigación que confluyen en el problema e identifica la brecha que el trabajo aborda. El capítulo 4 describe la metodología: el caso de estudio, los datos, los *baselines* y el protocolo de evaluación. Los capítulos 5 a 8 presentarán el sistema implementado, los resultados, su discusión y las conclusiones.

2 Marco teórico

Este capítulo presenta los conceptos en los que se apoya el trabajo: la representación de la duración como variable aleatoria, el pronóstico por clases de referencia, el agrupamiento parcial, la simulación de Monte Carlo, la evaluación de pronósticos probabilísticos, la validación temporal, la no estacionariedad y la confiabilidad entre evaluadores.

2.1 La duración como variable aleatoria

La duración de una tarea de software no se conoce de antemano y puede representarse como una variable aleatoria, descrita por su distribución de probabilidad. Las duraciones tienen un límite inferior en cero, carecen de un límite superior definido y suelen presentar asimetría positiva, con una cola derecha extensa. Little reportó que, en 106 proyectos comerciales, la razón entre la duración real y la estimada sigue aproximadamente una distribución lognormal [11]. Una distribución simétrica, como la normal, no resulta adecuada para este tipo de variable, porque asigna la misma probabilidad a desvíos de igual magnitud por encima y por debajo del valor central.

El método de Rootstrap adopta la distribución de Weibull como familia de trabajo y prevé compararla fuera de muestra con la lognormal, la gamma, mezclas de distribuciones y el *bootstrap* empírico, de modo que la familia utilizada se decide con los datos y no a priori [4].

Un cuantil de orden p es el valor por debajo del cual se encuentra una proporción p de la distribución. Este trabajo utiliza tres cuantiles, siguiendo la convención del método de Rootstrap: el P50 o mediana, el P80 y el P95. Cada uno se asocia a una decisión distinta. El P50 es la fecha con igual probabilidad de cumplirse o no, y es la que se usa para planificar; el P80 es la fecha que se compromete ante el cliente; y el P95 delimita el riesgo que se incorpora al precio de un contrato a precio fijo [4].

2.2 Pronóstico por clases de referencia

El pronóstico por clases de referencia (*reference class forecasting*) estima un caso nuevo a partir de la distribución de resultados de casos anteriores comparables, en lugar de razonar desde los detalles particulares del caso. Flyvbjerg lo presentó como la aplicación práctica de la visión externa (*outside view*) propuesta por Kahneman y Tversky, como corrección del sesgo optimista que produce la visión interna [12], y documentó su adopción en la planificación de proyectos de infraestructura [13].

El problema central del método es la construcción de la clase. Una revisión sistemática de 61 artículos concluye que la utilidad del pronóstico depende en gran medida de cómo se define la clase de referencia, y describe un compromiso entre amplitud y homogeneidad: una clase amplia reúne muchos casos, pero heterogéneos, mientras que una clase estrecha es más homogénea, pero contiene pocos casos [14]. La misma revisión reporta la evolución del método hacia variantes híbridas que tratan la clase de referencia como distribución a priori de un modelo bayesiano [14].

El método de Rootstrap aplica esta idea a nivel de tarea: cada *capability* se clasifica en una grilla de cinco niveles de complejidad (C1 a C5) y dos tamaños (S y L), y cada celda de la grilla funciona como una clase de referencia [4]. Con entre 100 y 300 tareas repartidas en diez celdas, es esperable que algunas reúnan muy pocos casos, lo que vuelve poco confiable cualquier estimación de sus cuantiles extremos.

2.3 Agrupamiento parcial

El agrupamiento parcial (*partial pooling*) ofrece una solución intermedia entre estimar cada clase por separado y agrupar todas las clases en una sola. La estimación de cada clase se contrae (*shrinkage*) hacia la de un grupo más amplio, con un peso que depende de la cantidad de información propia: una clase con muchas observaciones conserva su estimación, mientras que una clase con pocas se apoya en las demás [15].

El método de Rootstrap adopta este enfoque en escala logarítmica. El valor de referencia inicial de cada celda se trata como un número fijo de pseudoobservaciones, que se combina con las duraciones observadas en un promedio ponderado por la cantidad de información de cada fuente [4]. A medida que se acumulan entregas, la estimación de la celda se desplaza desde el valor de referencia hacia el valor medido.

2.4 Simulación de Monte Carlo

La duración de un proyecto no es la suma de las duraciones de sus tareas. Las tareas se ejecutan en paralelo cuando los recursos lo permiten, esperan a que terminen aquellas de las que dependen, y la duración total queda determinada por la cadena de dependencias que termina en último lugar. La composición de distribuciones asimétricas sobre un grafo de dependencias con recursos limitados no tiene, en general, una solución analítica, y por ese motivo se recurre a la simulación. La simulación de redes de actividades mediante Monte Carlo tiene antecedentes en la planificación de proyectos desde la década de 1960 [16].

El método de Monte Carlo [17] consiste, en este contexto, en ejecutar el proyecto de forma simulada muchas veces. En cada corrida se sortea una duración para cada tarea a partir

de la distribución de su clase, se programan las tareas respetando las dependencias y la cantidad de líneas de trabajo paralelas disponibles, y se registra la fecha de finalización. Con 10.000 corridas, los cuantiles de la distribución de fechas se obtienen directamente de los resultados ordenados: el P50 corresponde al valor en la posición 5.000, el P80 al de la posición 8.000 y el P95 al de la posición 9.500 [4]. La cadena de dependencias más larga fija un piso que ninguna cantidad de recursos adicionales permite reducir [4].

Una variante más simple remuestrea los tiempos históricos entre finalizaciones sucesivas, sin distinguir categorías ni dependencias. Miranda et al. evaluaron esa variante con datos reales de una organización y obtuvieron una MMRE de 32 % en la predicción de la fecha de entrega de diez proyectos [3].

2.5 Evaluación de pronósticos probabilísticos

2.5.1 Calibración y nitidez

Un pronóstico probabilístico se evalúa sobre un conjunto de pronósticos y sus resultados, y no sobre un caso aislado. Que una tarea supere su P80 no indica por sí solo un error, porque esa situación debe ocurrir en alrededor de uno de cada cinco casos.

Gneiting, Balabdaoui y Raftery distinguen dos propiedades [7]. La calibración es la consistencia estadística entre las distribuciones pronosticadas y los resultados observados, y depende de ambos. La nitidez es la concentración de las distribuciones pronosticadas y depende solo del pronóstico. El criterio que proponen es maximizar la nitidez sujeto a la calibración: entre los pronósticos calibrados, se prefiere el más concentrado [7]. Jørgensen trasladó este criterio a la estimación de software como la maximización de la informatividad sujeta a la calibración [18]. Una revisión general de estas herramientas se encuentra en [19].

Los mismos autores muestran, mediante una simulación, que cuatro pronosticadores de calidad muy distinta producen coberturas prácticamente idénticas del intervalo central de 50 % (entre 50,1 % y 51,3 %) e histogramas de la transformada integral de probabilidad casi uniformes; lo que los distingue es el ancho de sus intervalos y su puntuación en reglas propias [7]. Esta observación tiene una consecuencia directa para este trabajo: un *baseline* que utiliza la misma distribución para todas las tareas puede estar bien calibrado y ser, aun así, mucho menos informativo que un modelo que condiciona en la categoría.

2.5.2 Cobertura de cuantiles

La cobertura empírica de un cuantil es la proporción de casos en que el valor observado quedó por debajo del cuantil pronosticado, y constituye la forma operativa de la calibración probabilística [7]. En un sistema calibrado, la cobertura empírica del P80 se aproxima a 80 %.

Una cobertura menor indica sobreconfianza, es decir, intervalos más estrechos que la incertidumbre real; una cobertura mayor indica un pronóstico conservador.

2.5.3 Transformada integral de probabilidad

La transformada integral de probabilidad (PIT, por sus siglas en inglés) resume la calibración de la distribución completa y no de un único cuantil. Para cada observación se calcula el valor de la función de distribución pronosticada en el resultado observado, es decir, el percentil de la distribución en que cayó el valor real. Si el pronóstico es ideal y la distribución es continua, esos valores se distribuyen de forma uniforme entre 0 y 1 [7].

Las desviaciones de la uniformidad tienen una lectura diagnóstica. Un histograma con forma de U indica distribuciones demasiado estrechas; una acumulación en los valores centrales indica distribuciones demasiado amplias; y una forma triangular indica un sesgo [7].

Este resultado supone distribuciones continuas. Cuando las duraciones se registran en unidades enteras, como días, la distribución pronosticada presenta saltos y la PIT no resulta uniforme aunque el modelo sea correcto. Czado, Gneiting y Held propusieron una versión no aleatorizada de la PIT para datos discretos [20].

2.5.4 Reglas de puntuación propias

Una regla de puntuación asigna un valor numérico a cada par formado por una distribución pronosticada y un resultado observado. Una regla es propia cuando la puntuación esperada se optimiza al declarar la distribución que efectivamente se considera correcta, de modo que no existe incentivo para distorsionarla [21].

El puntaje de probabilidad clasificado continuo (CRPS, por sus siglas en inglés) es una regla propia que mide la distancia entre la función de distribución pronosticada F y el valor observado x :

$$CRPS(F, x) = \int [F(y) - 1\{y \geq x\}]^2 dy$$

donde $1\{y \geq x\}$ vale 1 cuando $y \geq x$ y 0 en caso contrario. El CRPS se expresa en las mismas unidades que la variable pronosticada y se reduce al error absoluto cuando el pronóstico es un único valor [7], [21]. Esta última propiedad permite comparar en una misma escala un modelo probabilístico y un *baseline* que produce una fecha puntual. La pérdida *pinball*, o puntuación de cuantiles, evalúa cada cuantil por separado y permite identificar en qué región de la distribución se concentran los errores [21].

2.6 Validación temporal y fuga de información

Evaluar un modelo con los mismos datos con los que se ajustó mide su capacidad de reproducir el pasado y no su capacidad de pronosticar. Cuando los datos tienen un orden temporal, la práctica recomendada es la evaluación con origen móvil (*rolling origin*): se fija un momento, se ajusta el modelo con la información disponible hasta ese momento, se pronostica lo que ocurre después, se registra el resultado y se avanza el origen [22].

En ingeniería de software, Sigweni, Shepperd y Turchi compararon la validación cruzada dejando uno afuera con una validación cronológica sobre 477 predicciones en dos conjuntos de datos [23]. A partir de sus resultados, la validación cronológica produjo errores medios 13,7 % y 48,1 % más altos en cada conjunto. En uno de ellos, la interacción entre el esquema de validación y la técnica de estimación fue significativa ($p = 0,046$), lo que indica que el esquema de validación puede alterar el orden de preferencia entre modelos [23].

La fuga de información (*leakage*) ocurre cuando el modelo utiliza, para pronosticar, información que no existía en el momento del pronóstico. Puede producirse al entrenar con tareas que terminaron después de ese momento, al usar categorías asignadas después de conocer el resultado, al normalizar los datos con estadísticos calculados sobre todo el histórico o al elegir hiperparámetros observando el período de evaluación. La regla que adopta este trabajo es que, en cada paso de la evaluación, el modelo solo puede utilizar información disponible en el momento del pronóstico.

2.7 No estacionariedad

Un proceso es no estacionario cuando la relación entre las variables y el resultado cambia con el tiempo. En la literatura de aprendizaje automático este fenómeno se denomina deriva de concepto (*concept drift*), y las estrategias de adaptación incluyen ventanas que descartan los datos antiguos y mecanismos que ponderan los datos según su antigüedad [24].

En un sistema de entrega ejecutado por agentes, la no estacionariedad es esperable por diseño: cuando cambian el modelo de lenguaje, las instrucciones o el ciclo de trabajo, las tareas anteriores fueron ejecutadas por un proceso distinto del actual. El método de Rootstrap registra la versión del proceso en cada ejecución, segmenta los ajustes por versión y pondera las observaciones por recencia mediante un decaimiento exponencial [4].

Minku y Yao proponen una alternativa a descartar la historia antigua: mantener modelos entrenados en distintos períodos y ponderarlos según su desempeño reciente. En su análisis de componentes, la ponderación dinámica explica la mejora obtenida, mientras que el filtrado por similitud no aporta una diferencia significativa [25].

2.8 Confiabilidad entre evaluadores

Si la categoría de una tarea se asigna después de conocer su duración, el modelo resulta circular: las tareas largas tienden a clasificarse como complejas y el modelo encuentra luego que las tareas complejas demoran más. Fischhoff documentó que conocer el resultado de un evento aumenta la probabilidad que se le atribuye a posteriori [26]. Por ese motivo, cuando las categorías no se registraron antes de la ejecución, deben asignarse a ciegas, sin acceso al resultado.

La confiabilidad de una clasificación se mide por el acuerdo entre evaluadores independientes, corregido por el acuerdo esperable por azar. El coeficiente kappa de Cohen trata las categorías como nominales. Para evaluaciones de procesos de software, El Emam propuso umbrales derivados de 70 valores empíricos de kappa: por debajo de 0,45 el acuerdo se considera pobre, entre 0,45 y 0,62 moderado, entre 0,63 y 0,78 sustancial y por encima de 0,78 excelente [27]. Cuando las categorías tienen un orden, como los niveles de complejidad C1 a C5, el alfa de Krippendorff permite tratar la escala como ordinal e incluir más de dos evaluadores [28]; los umbrales de El Emam no se aplican directamente a ese caso.

3 Estado del arte

El problema de este trabajo se ubica en el cruce de cinco líneas de investigación que, en la literatura revisada, casi no se citan entre sí: la estimación en ingeniería de software, la evaluación de pronósticos probabilísticos, la validación temporal de modelos de estimación, el pronóstico por clases de referencia y la medición empírica del desarrollo con agentes de inteligencia artificial. Este capítulo revisa qué resolvió cada línea y qué dejó abierto, y cierra con la brecha que ocupa el trabajo.

3.1 Estimación en ingeniería de software

Shepperd y MacDonell señalan que buena parte de la literatura de estimación evalúa estimaciones puntuales con medidas de error como la MMRE, cuestionan esa práctica y proponen comparar siempre los sistemas de predicción con un *baseline* trivial e informar el tamaño del efecto [29]. La advertencia tiene respaldo empírico: Tawosi, Moussa y Sarro replicaron un modelo de aprendizaje profundo para estimar puntos de historia sobre 31.960 *issues* y encontraron que apenas supera a la mediana [30]. Menzies et al. reportaron, en cuatro conjuntos de datos, que métodos más recientes no superaron de forma consistente al modelo COCOMO original [10].

La simulación de Monte Carlo sobre datos históricos cuenta con al menos una evaluación con datos reales en software. Miranda et al. remuestrearon los tiempos entre finalizaciones de historias de usuario, sin categorías ni dependencias, y obtuvieron una MMRE de 32 % en la fecha de entrega de diez proyectos y de 20 % en el esfuerzo de cinco proyectos, frente a 134 % de las estimaciones de los desarrolladores [3]. El estudio emite intervalos de predicción al 95 %, pero no evalúa su cobertura. Un cálculo propio a partir de sus tablas indica que el valor real quedó dentro del intervalo en 11 de 15 casos, una cobertura de 73 %; con esa cantidad de casos la observación no es concluyente, aunque es consistente con intervalos demasiado estrechos. Los autores observaron además que el error aumenta cuando el historial utilizado supera las 60 historias de usuario, lo que atribuyeron a ruido; la no estacionariedad del proceso es una explicación alternativa [3].

Jørgensen y Sjøberg propusieron construir intervalos de predicción a partir de la distribución empírica del error de estimaciones anteriores, agrupando tareas con una incertidumbre esperada similar. Sobre 145 tareas de mantenimiento, el método empírico alcanzó una cobertura de 85 % para un nivel nominal de 90 %, con intervalos más estrechos

que los de los métodos paramétricos [2]. Los autores advierten que utilizaron validación cruzada en lugar del orden temporal real de las tareas [2].

Esta línea deja dos cuestiones abiertas. La evaluación se concentra en errores puntuales o en la tasa de acierto de intervalos, sin reglas de puntuación propias ni análisis de la distribución completa. Y los métodos que emiten intervalos, como la simulación de Monte Carlo por remuestreo, no reportan su calibración.

3.2 Evaluación de pronósticos probabilísticos en proyectos

El paradigma de calibración y nitidez, la PIT y las reglas de puntuación propias se desarrollaron en la literatura estadística, con aplicaciones principalmente meteorológicas y económicas [7], [19]. La aplicación más cercana a este trabajo es la de Jørgensen, Welde y Halkjelsvik, que evaluaron estimaciones probabilísticas de costo de 69 proyectos públicos noruegos con la tasa de acierto, la PIT, el ancho relativo de los intervalos y el CRPS [9]. El P85 cubrió el costo real en 75 % de los casos, y el histograma de la PIT presentó exceso en ambos extremos, señal de distribuciones demasiado estrechas.

Los mismos autores construyeron un estimador hipotético que aplica una única distribución a todos los proyectos. Ese estimador obtuvo mejor CRPS (mediana de 88 frente a 109) y mejor cobertura que las estimaciones reales, pero requería el doble de contingencia y no distinguía los proyectos más riesgosos [9]. Esa construcción corresponde al *baseline* de conteo puro de este trabajo, y anticipa que un empate en CRPS entre el modelo y ese *baseline* debe interpretarse junto con medidas de informatividad.

El trabajo evalúa el costo de proyectos públicos y no la duración de tareas de software, y lo hace sobre observaciones prácticamente independientes, sin origen móvil. En la literatura revisada no se encontró una aplicación de este marco a la entrega de software.

3.3 Validación temporal y no estacionariedad en estimación

La pregunta sobre cuánta historia utilizar tiene resultados contradictorios. Lokan y Mendes estudiaron el uso de ventanas móviles en la estimación de esfuerzo [31] y, en una réplica posterior, no encontraron mejoras en la precisión [32]. Minku y Yao abordaron el problema con modelos entrenados en distintos períodos que se ponderan de forma dinámica, y su método superó a seis alternativas en cuatro conjuntos de datos [25].

Estos trabajos operan a nivel de proyecto, sobre conjuntos de datos que combinan varias organizaciones, con el tiempo como variable continua y sin una variable explícita de versión del proceso. El caso de este trabajo presenta características distintas: la unidad es la tarea, el

proceso pertenece a una sola organización y sus cambios quedan fechados por la versión registrada en la telemetría.

En cuanto al esquema de validación, Sigweni et al. identifican un límite de su propia propuesta: los proyectos se ordenan por fecha de finalización, de modo que un proyecto entra al conjunto de entrenamiento cuando termina, aunque su pronóstico se haga al inicio; los autores lo señalan como trabajo futuro [23]. Este trabajo adopta un criterio más estricto, que se describe en el capítulo 4.

3.4 Clases de referencia y taxonomías de trabajo

El pronóstico por clases de referencia se aplicó sobre todo a megaproyectos de infraestructura, transporte y energía [13], [14]. La revisión de Cantarelli et al. se basa casi por completo en proyectos de infraestructura, construcción y energía, y señala como problema abierto la construcción de la clase, incluida la variabilidad que introduce el juicio de quien la define [14].

Sobre la confiabilidad de las categorías de trabajo, Herzig, Just y Zeller encontraron que alrededor de un tercio de los *issues* registrados como errores en los sistemas de seguimiento no lo eran [33], y El Emam derivó umbrales de acuerdo para evaluaciones de procesos de software [27]. No se encontraron estudios que midan el acuerdo entre evaluadores al asignar niveles de complejidad o tamaño a tareas de software.

3.5 Medición empírica del desarrollo con agentes

La investigación empírica sobre agentes de código creció en 2026 alrededor del conjunto de datos AIDev, que reúne *pull requests* abiertos por agentes autónomos en repositorios públicos [5]. Su subconjunto enriquecido incluye la línea de tiempo de eventos de cada *pull request*, lo que permite reconstruir duraciones, aunque el artículo no reporta estadísticos temporales [5].

Dao et al. predijeron, con información disponible al crear el *pull request*, cuáles caerían en el 20 % de mayor esfuerzo de revisión, con un área bajo la curva ROC de 0,96 en una partición temporal y de 0,83 en repositorios no vistos durante el entrenamiento [8]. Reportan una curva de calibración para esa probabilidad binaria, sin una métrica numérica asociada, y observan que el 28,3 % de los *pull requests* se integra en menos de un minuto [8]. En el estudio de campo de Kumar et al., los autores declaran explícitamente que no analizaron la duración de las actividades [6].

El trabajo que plantea de forma más directa la estimación en la ingeniería de software con agentes es el modelo ACEM de El-Ramly, un modelo determinístico de costo que, según su

resumen, no presenta validación con datos y cuyo autor señala la necesidad de una calibración empírica futura [34].

Esta línea mide la adopción, la productividad y la intervención humana, pero no la entrega de proyectos. No se encontraron trabajos que pronostiquen fechas de entrega de trabajo ejecutado por agentes ni que evalúen la calibración de distribuciones predictivas sobre ese trabajo. La evidencia disponible proviene además, casi en su totalidad, de repositorios de código abierto, sin latencia de decisiones de clientes, escalaciones formales ni versión del proceso.

3.6 Síntesis y brecha

Cada línea resolvió una parte del problema. La estimación en software aporta el motor de simulación y *baselines* con respaldo publicado; la literatura de pronóstico probabilístico aporta el criterio de evaluación y sus métricas; la validación temporal aporta la disciplina para evaluar sin fuga de información; el pronóstico por clases de referencia aporta la estructura del modelo y sus riesgos; y la medición del trabajo con agentes describe el fenómeno sin pronosticarlo.

La brecha requiere una formulación precisa. La calibración de probabilidades binarias sobre trabajo agéntico existe [8]. Lo que no se encontró en la literatura revisada es la evaluación de la calibración de distribuciones predictivas sobre la duración o la fecha de entrega de software, con equipos humanos o con agentes. En particular, quedan sin abordar:

- el pronóstico probabilístico de fechas de entrega para trabajo ejecutado por agentes;
- la evaluación de la calibración de la simulación de Monte Carlo sobre datos históricos de software, cuyos intervalos se emiten sin verificarse [3];
- la confiabilidad entre evaluadores al asignar niveles de complejidad y tamaño a tareas de software;
- el uso de ventanas y de ponderación por recencia a nivel de tarea, en una organización y con cambios de proceso identificados.

Este trabajo se ubica en esa intersección: aplica los criterios de evaluación probabilística a un motor de pronóstico basado en clases de referencia, sobre datos de desarrollo con agentes y con una validación temporal estricta.

4 Metodología

Este capítulo describe el diseño de la investigación. De acuerdo con el compromiso asumido en el anteproyecto, el diseño de la evaluación se especifica antes de observar resultados. Las decisiones que dependen de las características de los datos se identifican como pendientes.

4.1 Enfoque

El trabajo sigue la vía de producto de la maestría: el entregable es un sistema en funcionamiento, con un usuario y una decisión comercial concretos, y la investigación se integra en él como capa de evaluación. Desde el punto de vista metodológico, se trata de un estudio empírico retrospectivo sobre datos observacionales, en el que los pronósticos se reconstruyen para momentos pasados y se comparan con los resultados que efectivamente ocurrieron.

4.2 Caso de estudio y fuente de datos

Los datos provienen de Rootstrap, una empresa de desarrollo de software que ejecuta sus proyectos mediante un sistema de entrega basado en agentes. En ese sistema, cada proyecto sigue el mismo ciclo (planificación, construcción, revisión adversarial, integración y convergencia de control de calidad), ejecutado por agentes bajo controles humanos y contra una especificación completa y validada que la empresa denomina *Ground Truth* [4]. El término no tiene aquí el sentido habitual en aprendizaje automático: designa la especificación autoritativa del software a construir.

La unidad de trabajo es la *capability*, una función de negocio que puede cotizarse y entregarse de forma independiente, compuesta por historias de usuario con criterios de aceptación [4]. Cada *capability* se clasifica según su nivel de complejidad y su tamaño en el mismo punto de control humano que aprueba su especificación. La Tabla 4.1 resume la grilla y los valores de referencia iniciales que propone el método; esos valores son ilustrativos y se corrigen con los datos observados [4].

Tabla 4.1. Clases de referencia del método de Rootstrap y valores iniciales ilustrativos, expresados como mediana del tiempo de trabajo transcurrido. Elaboración propia a partir de [4].

Nivel	Naturaleza de la complejidad	Tamaño S	Tamaño L
C1	Flujo único, con patrones conocidos y reglas determinísticas	~1 día	~2–3 días

Nivel	Naturaleza de la complejidad	Tamaño S	Tamaño L
C2	Varios flujos, con reglas de negocio y estados de falla	~3–4 días	~1 semana
C3	Transversal, con sistemas externos no controlados	~1 semana	~2 semanas
C4	Plataforma o componente probabilístico, con evaluación extensa	~1–2 semanas	~2–3 semanas
C5	Estratégico: irreversible, regulado o novedoso	~3 semanas	Se divide en varias <i>capabilities</i>

Según la descripción del método, la telemetría registra las duraciones de cada entrega, los rebotes en la convergencia de control de calidad, las marcas de tiempo de cada decisión solicitada al cliente, el estado de la especificación, la persona y el agente responsables de cada ejecución y la versión del proceso que produjo cada unidad de trabajo [4]. El trabajo utilizará un export anonimizado de alrededor de diez proyectos históricos, bajo un acuerdo escrito con la empresa. [PENDIENTE: confirmar los campos del export, en particular si incluye la descripción de cada tarea, una estimación puntual por tarea, las horas de intervención humana y la resolución temporal de las duraciones.]

Como fuente alternativa se considera el conjunto de datos AIDev [5], que permite reconstruir, para cada *pull request*, el momento de creación y el de integración o cierre. Su uso implicaría cambiar la pregunta de investigación, ya que la unidad sería un *pull request* de código abierto y no una tarea especificada de antemano en un proyecto comercial.

4.3 Unidad de análisis

La unidad primaria de análisis es la *capability*. Con alrededor de diez proyectos, la calibración a nivel de proyecto no puede validarse estadísticamente, porque cada proyecto aporta una única observación de su fecha de finalización. A nivel de tarea, cada proyecto aporta entre diez y treinta observaciones [4], lo que permite reunir del orden de 100 a 300. El nivel de proyecto se reporta como demostración, mediante una validación que deja un proyecto afuera en cada iteración, y no como validación estadística.

4.4 Etiquetado de las tareas

Si el histórico no registra la categoría de cada tarea antes de su ejecución, la categoría se asignará a ciegas. Dos o más personas clasificarán cada tarea de forma independiente, a partir de su especificación y sin acceso a su duración ni a su resultado, con el protocolo que describe El Emam: la misma evidencia para todos los evaluadores, clasificación sin discusión previa y

consenso posterior [27]. Este ejercicio habilita además el estudio de confiabilidad entre evaluadores planteado en el OE5(a).

4.5 Baselines

El modelo se comparará con tres alternativas especificadas de antemano:

1. **Conteo puro.** Todas las tareas se tratan como intercambiables y se remuestrea la distribución histórica global, sin categorías ni dependencias. Corresponde al método evaluado por Miranda et al. [3] y al estimador global que, según Jørgensen, Welde y Halkjelsvik, puede obtener un buen CRPS sin ser informativo [9].
2. **Distribuciones por categoría sin aprendizaje.** Se utiliza la tabla de valores de referencia por clase, sin actualización con los datos observados, en la línea del método empírico de Jørgensen y Sjøberg [2]. Ese método requiere una estimación puntual previa por tarea. [PENDIENTE: si el export no la incluye, el baseline se anclará a la mediana de la clase.]
3. **Estimación humana,** en los casos en que exista registro.

4.6 Protocolo de evaluación

4.6.1 Backtesting con origen móvil

Los pronósticos se reconstruirán con origen móvil. El corte de cada paso es el momento del pronóstico, y una tarea entra al conjunto de entrenamiento solo si su desenlace ya se conocía en ese momento. Este criterio es más estricto que el de Sigweni et al., que ordenan las observaciones por fecha de finalización [23]. El corte es temporal y no por proyecto, para que las tareas de un mismo proyecto no se transfieran información entre sí.

La exactitud se calificará contra el alcance vigente en el momento del pronóstico y no contra el plan actualizado con cambios posteriores, para que la incorporación de trabajo nuevo no se confunda con error del modelo [4].

4.6.2 Métricas

Para cada modelo se calcularán las siguientes medidas:

- la cobertura empírica de los cuantiles P50, P80 y P95;
- el histograma de la PIT, en su versión no aleatorizada si las duraciones son discretas [20];
- el CRPS, expresado en días, y la pérdida *pinball* de cada cuantil [21];
- el ancho del intervalo entre el P50 y el P95, como medida de nitidez;

- la correlación entre el ancho de los intervalos y el error absoluto, como medida de informatividad [9].

La mediana pronosticada se evaluará con el error absoluto o con el error absoluto en escala logarítmica, y no con la MMRE, porque la función de pérdida implícita en la métrica debe minimizarse en el punto que se evalúa [9].

4.6.3 Criterio de comparación

La calibración se utilizará como condición y no como criterio de ordenamiento: entre los modelos calibrados, se preferirá el de menor CRPS y mayor nitidez [7]. La uniformidad de la PIT se contrastará con una prueba robusta a muestras pequeñas y a la correlación entre valores sucesivos que introduce el origen móvil [35], [36]. Las diferencias de CRPS entre modelos se contrastarán con la prueba de Diebold y Mariano, con la corrección para muestras pequeñas de Harvey, Leybourne y Newbold [37], [38]. El cálculo del CRPS se apoyará en implementaciones publicadas [39]. [PENDIENTE: elegir entre la prueba de Knüppel y la de Rossi y Sekhposyan.]

Para reducir el sesgo de análisis, quien ejecute los modelos no será quien analice los resultados, siguiendo el protocolo de análisis ciego de Sigweni et al. [23].

4.6.4 Validación complementaria

Sobre los mismos modelos se ejecutará además una validación cruzada aleatoria, y se reportará la diferencia con el origen móvil. El diseño replica el de Sigweni et al. [23], pero expresa la diferencia en términos de calibración y no solo de error puntual.

4.7 Estudios del OE5

Para la confiabilidad entre evaluadores, OE5(a), se medirá el acuerdo en la asignación del nivel y del tamaño por separado, con el kappa de Cohen para la dimensión binaria y el alfa de Krippendorff ordinal para los niveles [28]. [PENDIENTE: decidir antes del estudio si los niveles se tratan como nominales, caso en que se aplican los umbrales de El Emam.]

Para la cantidad de historia, OE5(b), se comparará el desempeño con distintas ventanas y ponderaciones por recencia bajo el mismo protocolo de origen móvil, con la hipótesis nula de que la cantidad de historia utilizada no afecta la calidad del pronóstico [25], [32]. [PENDIENTE: el estudio que se priorice se definirá con la tutora.]

4.8 Consideraciones éticas y de datos

El trabajo no procesa datos personales ni información identificable de clientes. La telemetría se recibe anonimizada, bajo un acuerdo escrito que incluye la autorización para publicar los resultados, sean favorables o no. [PENDIENTE: confirmar el estado del acuerdo con Rootstrap y la autorización para citar su documento técnico.]

Referencias

- [1] M. Jørgensen and M. Shepperd, “A systematic review of software development cost estimation studies,” *IEEE Trans. Softw. Eng.*, vol. 33, no. 1, pp. 33–53, Jan. 2007, doi: 10.1109/TSE.2007.256943.
- [2] M. Jørgensen and D. I. K. Sjøberg, “An effort prediction interval approach based on the empirical distribution of previous estimation accuracy,” *Inf. Softw. Technol.*, vol. 45, no. 3, pp. 123–136, Mar. 2003, doi: 10.1016/S0950-5849(02)00188-X.
- [3] P. Miranda, J. P. Faria, F. F. Correia, A. Fares, R. Graça, and J. Mendes Moreira, “An analysis of Monte Carlo simulations for forecasting software projects,” in *Proc. 36th Annu. ACM Symp. Appl. Comput. (SAC)*, 2021, pp. 1550–1558, doi: 10.1145/3412841.3442030.
- [4] Rootstrap, “Estimations: About to be solved. Data-backed, semi-deterministic software estimation on a delivery system,” Rootstrap, White paper, Aug. 2026.
- [5] H. Li, H. Zhang, and A. E. Hassan, “AIDev: Studying AI coding agents on GitHub,” 2026, *arXiv:2602.09185*.
- [6] A. Kumar, Y. Bajpai, S. Gulwani, G. Soares, and E. Murphy-Hill, “Why AI agents still need you: Findings from developer-agent collaborations in the wild,” in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2025, arXiv:2506.12347.
- [7] T. Gneiting, F. Balabdaoui, and A. E. Raftery, “Probabilistic forecasts, calibration and sharpness,” *J. R. Stat. Soc. Ser. B Stat. Methodol.*, vol. 69, no. 2, pp. 243–268, Apr. 2007, doi: 10.1111/j.1467-9868.2007.00587.x.
- [8] S. D. M. Dao et al., “Early-stage prediction of review effort in AI-generated pull requests,” in *Proc. 23rd Int. Conf. Mining Softw. Repositories (MSR)*, Rio de Janeiro, Brazil, 2026, doi: 10.1145/3793302.3793609.
- [9] M. Jørgensen, M. Welde, and T. Halkjelsvik, “Evaluation of probabilistic project cost estimates,” *IEEE Trans. Eng. Manag.*, vol. 70, no. 10, pp. 3481–3496, Oct. 2023, doi: 10.1109/TEM.2021.3067050.
- [10] T. Menzies, Y. Yang, G. Mathew, B. Boehm, and J. Hihn, “Negative results for software effort estimation,” *Empirical Softw. Eng.*, vol. 22, no. 5, pp. 2658–2683, Oct. 2017, doi: 10.1007/s10664-016-9472-2.
- [11] T. Little, “Schedule estimation and uncertainty surrounding the cone of uncertainty,” *IEEE Softw.*, vol. 23, no. 3, pp. 48–54, May 2006, doi: 10.1109/MS.2006.82.
- [12] B. Flyvbjerg, “From Nobel Prize to project management: Getting risks right,” *Project Manage. J.*, vol. 37, no. 3, pp. 5–15, Aug. 2006, doi: 10.1177/875697280603700302.
- [13] B. Flyvbjerg, “Curbing optimism bias and strategic misrepresentation in planning: Reference class forecasting in practice,” *Eur. Planning Stud.*, vol. 16, no. 1, pp. 3–21, Jan. 2008, doi: 10.1080/09654310701747936.

- [14] C. C. Cantarelli, K. Davis, J. K. Pinto, and N. Turner, "Reference class forecasting: Promises, problems, and a research agenda moving forward," *Prod. Planning Control*, vol. 37, no. 7, pp. 691–709, May 2026, doi: 10.1080/09537287.2025.2578708.
- [15] B. Efron and C. Morris, "Data analysis using Stein's estimator and its generalizations," *J. Amer. Statist. Assoc.*, vol. 70, no. 350, pp. 311–319, Jun. 1975, doi: 10.1080/01621459.1975.10479864.
- [16] R. M. Van Slyke, "Monte Carlo methods and the PERT problem," *Oper. Res.*, vol. 11, no. 5, pp. 839–860, Oct. 1963, doi: 10.1287/opre.11.5.839.
- [17] N. Metropolis and S. Ulam, "The Monte Carlo method," *J. Amer. Statist. Assoc.*, vol. 44, no. 247, pp. 335–341, Sep. 1949, doi: 10.1080/01621459.1949.10483310.
- [18] M. Jørgensen, "Evaluating probabilistic software development effort estimates: Maximizing informativeness subject to calibration," *Inf. Softw. Technol.*, vol. 115, pp. 93–96, Nov. 2019, doi: 10.1016/j.infsof.2019.08.006.
- [19] T. Gneiting and M. Katzfuss, "Probabilistic forecasting," *Annu. Rev. Stat. Appl.*, vol. 1, pp. 125–151, Jan. 2014, doi: 10.1146/annurev-statistics-062713-085831.
- [20] C. Czado, T. Gneiting, and L. Held, "Predictive model assessment for count data," *Biometrics*, vol. 65, no. 4, pp. 1254–1261, Dec. 2009, doi: 10.1111/j.1541-0420.2009.01191.x.
- [21] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *J. Amer. Statist. Assoc.*, vol. 102, no. 477, pp. 359–378, Mar. 2007, doi: 10.1198/016214506000001437.
- [22] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: An analysis and review," *Int. J. Forecasting*, vol. 16, no. 4, pp. 437–450, Oct. 2000, doi: 10.1016/S0169-2070(00)00065-0.
- [23] B. Sigweni, M. Shepperd, and T. Turchi, "Realistic assessment of software effort estimation models," in *Proc. 20th Int. Conf. Eval. Assessment Softw. Eng. (EASE)*, Limerick, Ireland, 2016, pp. 1–6, doi: 10.1145/2915970.2916005.
- [24] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, Apr. 2014, doi: 10.1145/2523813.
- [25] L. L. Minku and X. Yao, "Which models of the past are relevant to the present? A software effort estimation approach to exploiting useful past models," *Automated Softw. Eng.*, vol. 24, no. 3, pp. 499–542, Sep. 2017, doi: 10.1007/s10515-016-0209-7.
- [26] B. Fischhoff, "Hindsight $\neq$ foresight: The effect of outcome knowledge on judgment under uncertainty," *J. Exp. Psychol.: Hum. Percept. Perform.*, vol. 1, no. 3, pp. 288–299, Aug. 1975, doi: 10.1037/0096-1523.1.3.288.
- [27] K. El Emam, "Benchmarking kappa: Interrater agreement in software process assessments," *Empirical Softw. Eng.*, vol. 4, no. 2, pp. 113–133, Jun. 1999, doi: 10.1023/A:1009820201126.
- [28] A. F. Hayes and K. Krippendorff, "Answering the call for a standard reliability measure for coding data," *Commun. Methods Meas.*, vol. 1, no. 1, pp. 77–89, Apr. 2007, doi: 10.1080/19312450709336664.
- [29] M. Shepperd and S. MacDonell, "Evaluating prediction systems in software project estimation," *Inf. Softw. Technol.*, vol. 54, no. 8, pp. 820–827, Aug. 2012, doi: 10.1016/j.infsof.2011.12.008.
- [30] V. Tawosi, R. Moussa, and F. Sarro, "Agile effort estimation: Have we solved the problem yet? Insights from a replication study," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 2677–2697, Apr. 2023, doi: 10.1109/TSE.2022.3228739.

- [31] C. Lokan and E. Mendes, “Applying moving windows to software effort estimation,” in *Proc. 3rd Int. Symp. Empirical Softw. Eng. Meas. (ESEM)*, 2009, pp. 111–122, doi: 10.1109/ESEM.2009.5316019.
- [32] C. Lokan and E. Mendes, “Investigating the use of moving windows to improve software effort prediction: A replicated study,” *Empirical Softw. Eng.*, vol. 22, no. 2, pp. 716–767, Apr. 2017, doi: 10.1007/s10664-016-9446-4.
- [33] K. Herzig, S. Just, and A. Zeller, “It’s not a bug, it’s a feature: How misclassification impacts bug prediction,” in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, San Francisco, CA, USA, 2013, pp. 392–401, doi: 10.1109/ICSE.2013.6606585.
- [34] M. El-Ramly, “ACEM: A cost estimation model for agentic software engineering,” 2026, *arXiv:2608.02582*.
- [35] M. Knüppel, “Evaluating the calibration of multi-step-ahead density forecasts using raw moments,” *J. Bus. Econ. Statist.*, vol. 33, no. 2, pp. 270–281, Apr. 2015, doi: 10.1080/07350015.2014.948175.
- [36] B. Rossi and T. Sekhposyan, “Alternative tests for correct specification of conditional predictive densities,” *J. Econometrics*, vol. 208, no. 2, pp. 638–657, Feb. 2019, doi: 10.1016/j.jeconom.2018.07.008.
- [37] F. X. Diebold, “Comparing predictive accuracy, twenty years later: A personal perspective on the use and abuse of Diebold–Mariano tests,” *J. Bus. Econ. Statist.*, vol. 33, no. 1, pp. 1–9, Jan. 2015, doi: 10.1080/07350015.2014.983236.
- [38] D. Harvey, S. Leybourne, and P. Newbold, “Testing the equality of prediction mean squared errors,” *Int. J. Forecasting*, vol. 13, no. 2, pp. 281–291, Jun. 1997, doi: 10.1016/S0169-2070(96)00719-4.
- [39] A. Jordan, F. Krüger, and S. Lerch, “Evaluating probabilistic forecasts with scoringRules,” *J. Statist. Softw.*, vol. 90, no. 12, 2019, doi: 10.18637/jss.v090.i12.